

Distributional State-Corrected Action Suppression (Dist-SCAS): A Unified Framework for Risk-Sensitive Offline Reinforcement Learning

1. Introduction: The Convergence of Safety, Distribution, and Dynamics

The domain of Offline Reinforcement Learning (RL) stands at a pivotal intersection of statistical learning theory, optimal control, and generative modeling. The fundamental promise of offline RL—the ability to learn optimal policies from static, previously collected datasets without the risks or costs of online interaction—is tempered by the formidable challenge of distributional shift. When an agent attempts to maximize a value function learned from finite data, it inevitably encounters the "extrapolation error," where the value estimator erroneously assigns high utility to actions or states outside the support of the training distribution (Out-of-Distribution, or OOD). This phenomenon, often referred to as the "deadly triad" when combined with bootstrapping and function approximation, necessitates mechanisms that enforce conservatism.

Traditional approaches to enforcing conservatism have largely bifurcated into two distinct schools of thought: **value-based regularizers**, such as Conservative Q-Learning (CQL) or Implicit Q-Learning (IQL), which penalize Q-values for OOD actions $Q(s,a)$; and **policy-constraint methods**, such as Behavior Cloning (BC) or TD3+BC, which explicitly tether the learned policy to the behavior policy π_β . While effective, these scalar-value approaches often suffer from a reductionist view of uncertainty. They typically conflate *epistemic uncertainty* (uncertainty due to lack of data) with *aleatoric uncertainty* (inherent stochasticity in the environment). Consequently, a scalar penalty may suppress a highly promising action simply because the environment is noisy, or conversely, fail to penalize a confident but erroneous extrapolation in a sparse-data regime. ***

This report proposes and details the implementation of a novel, comprehensive framework: **Distributional State-Corrected Action Suppression (Dist-SCAS)**. This method synthesizes the geometric state-correction capabilities of the recently proposed SCAS framework ¹ with the statistical robustness of Distributional Reinforcement Learning, specifically focusing on lower-quantile maximization. ² ***

The SCAS framework addresses a critical gap in prior literature: the distinction between OOD actions and OOD states. While CQL effectively suppresses OOD actions on known states, it provides no guarantee that a valid action will not transition the agent into an OOD region of the state space where the value function is undefined. SCAS introduces a model-based regularizer to align the agent's induced transition dynamics with a "Value-Aware State Transition" distribution derived from the dataset. ³ ***

Concurrently, Distributional RL posits that modeling the full distribution of returns, $\mathbf{Z}(s,a)$, rather than merely the expectation $\mathbf{Q}(s,a)$, provides a richer signal for decision-making. ⁴ By optimizing a policy to maximize a lower quantile (e.g., the 10th percentile or $\tau=0.1$) of the return distribution, the agent naturally avoids actions with heavy-tailed downside risks—often a proxy for OOD uncertainty—without the need for the computationally expensive negative sampling required by CQL. ⁵ ***

Dist-SCAS represents a unified theory of "Safety through Geometry and Statistics." By rigorously deriving the mathematical interaction between the SCAS state-correction term and the distributional Bellman operator, this report offers a blueprint for an algorithm that is not only robust to OOD actions and states but also capable of disentangling risk from opportunity. The following sections provide an exhaustive analysis of

the theoretical underpinnings, a detailed derivation of the requisite loss functions, a comprehensive guide to code implementation including network architectures and hyperparameter selection, and a strategic validation plan utilizing the D4RL benchmark suite.

2. Theoretical Foundations and Literature Synthesis

To understand the necessity and mechanics of Dist-SCAS, one must first deconstruct the limitations of current state-of-the-art methods and the specific theoretical advances offered by SCAS and Distributional RL.

2.1 The Pathology of Extrapolation Error in Scalar Offline RL

The core objective of standard RL is to maximize the expected cumulative discounted return:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} [\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)]$$

In the offline setting, we only have access to a dataset $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{r}, \mathbf{s}')\}$ sampled from a behavior policy $\pi\beta$. Standard off-policy algorithms like Soft Actor-Critic (SAC) or TD3 approximate the Q-function by minimizing the Bellman error:

$$LQ(\theta) = \mathbb{E}_{(\mathbf{s}, \mathbf{a}, \mathbf{r}, \mathbf{s}') \sim \mathcal{D}} [(Q\theta(\mathbf{s}, \mathbf{a}) - (\mathbf{r} + \gamma Q\theta(\mathbf{s}', \pi(\mathbf{s}'))))^2]$$

The pathology arises in the maximization step $\pi(\mathbf{s}) = \arg\max_{\mathbf{a}} Q\theta(\mathbf{s}, \mathbf{a})$. **Since the Q-function is trained only on samples from $\pi\beta$** , its estimates for actions $\mathbf{a} \in \text{supp}(\pi\beta)$ are unconstrained and often arbitrarily high due to function approximation noise. The maximization operator exploits these errors, leading the policy towards OOD actions with hallucinated high values.

Conservative Q-Learning (CQL) addresses this by adding a regularizer that minimizes Q-values for OOD actions while maximizing them for data actions.

$$LCQL(\theta) = \alpha (\mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \mu(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] - \mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \pi\beta(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})]) + L_{\text{Bellman}}$$

While effective, CQL introduces a complex hyperparameter α (the penalty weight) and requires sampling from an approximate inverse distribution $\mu(\mathbf{a}|\mathbf{s})$, which is computationally expensive and difficult to tune. Furthermore, CQL operates on the expectation $\mathbb{E}[Q]$, ignoring the variance or shape of the value distribution, which contains critical information about safety.

2.2 The Case for Distributional Reinforcement Learning

Distributional RL replaces the scalar Q-function $Q(\mathbf{s}, \mathbf{a})$ with a random variable $\mathbf{Z}(\mathbf{s}, \mathbf{a})$ representing the full distribution of returns. The Distributional Bellman Operator $\mathbf{T}\pi$ is defined as:

$$\mathbf{Z}(\mathbf{s}, \mathbf{a}) = \mathcal{D}R(\mathbf{s}, \mathbf{a}) + \gamma \mathbf{Z}(\mathbf{s}', \mathbf{A}') \text{ where } \mathbf{A}' \sim \pi(\cdot|\mathbf{s}'), \mathbf{s}' \sim P(\cdot|\mathbf{s}, \mathbf{a})$$

The divergence between standard and distributional RL is profound in the offline setting.

Feature	Scalar RL (e.g., CQL, IQL)	Distributional RL (e.g., QR-DQN, TQC)
Value Estimate	Mean: $\mathbb{E}[\mathbf{Z}(\mathbf{s}, \mathbf{a})]$	Full Distribution: $P(\mathbf{Z}(\mathbf{s}, \mathbf{a}) \leq z)$
Uncertainty	Ignored or Implicit	Explicitly modeled via variance/quantiles

Feature	Scalar RL (e.g., CQL, IQL)	Distributional RL (e.g., QR-DQN, TQC)
Risk Sensitivity	Risk-Neutral	Configurable (e.g., CVaR, Lower Quantile)
OOD Handling	Requires explicit penalty terms	Naturally handled via pessimistic quantile selection

Lower Quantile Maximization: Recent works like Lower Expectile Q-learning (LEQ) ^{^^} and Truncated Quantile Critics (TQC) ^{^^} demonstrate that maximizing a lower quantile (e.g., the α -quantile where $\alpha \approx 0.1$) acts as a robust pessimism operator. If an action is OOD, the epistemic uncertainty regarding its outcome results in a high-variance return distribution. Consequently, the lower tail of this distribution drops significantly. By maximizing the lower tail, the agent naturally prefers actions where the outcome is both high-value and certain (low variance), effectively filtering OOD actions without explicit negative sampling.^{^^}

**** ****

2.3 The SCAS Framework: State Correction vs. Action Suppression

While Distributional RL addresses value uncertainty, it does not strictly prevent the agent from drifting into OOD states via valid actions—a subtle but critical distinction. **State Correction and Action Suppression (SCAS)** ^{^^} argues that OOD actions are merely the symptom; the disease is the OOD state visitation. **** ****

SCAS unifies state correction and action suppression by forcing the learned policy's induced transition distribution $\mathbf{P}_{\pi}(s'|s)$ to align with a "target" in-distribution (ID) transition $\mathbf{P}_{\mathbf{a}}(s'|s)$. This target distribution is derived from the dataset but re-weighted to favor high-value transitions. The SCAS objective is:
$$\max_{\pi} \mathbb{E} \left[\sum_t \gamma^t \left(Q(s_t, \pi(s_t)) - \lambda D_{\text{KL}}(\mathbf{P}^{\pi}(\cdot|s_t) \parallel \mathbf{P}_{\mathbf{a}}(\cdot|s_t)) \right) \right]$$
 The key insight from SCAS research is that by regularizing the *outcome* (next state) rather than just the *input* (action), the algorithm becomes robust to dynamics shifts and "delusional" policies that exploit model errors.^{^^} **** ****

3. Mathematical Framework of Dist-SCAS

Dist-SCAS merges these philosophies. We define the objective not as maximizing the expected return subject to state constraints, but as maximizing the **risk-adjusted return** (lower quantile) subject to **geometric state-correction constraints** .

3.1 The Distributional Bellman Operator under State Correction

Let the return distribution be approximated by a set of N quantiles, $\boldsymbol{\theta} = \{\theta_1, \dots, \theta_N\}$, such that $q_{\tau_i}(s, \mathbf{a}) = \theta_i$. The standard distributional projection minimizes the Wasserstein distance (approximated by Quantile Huber Loss) between the current distribution and the target distribution ****Y****:

$Y(s, \mathbf{a}) = r + \gamma Z(s', \pi(s'))$

In Dist-SCAS, the policy π used in the target calculation is the **Distributional-SCAS Policy** , which we denote **π_{dscas}** .

3.2 The Risk-Adjusted Objective

We define the risk-adjusted value **$Q_{\text{risk}}(s, \mathbf{a})$** as the Condition Value at Risk (CVaR) at level **α** :

$$Q_{risk}^{**}(s, a) = CVar\alpha^{**}(Z(s, a)) = E[Z(s, a) | Z(s, a) \leq F_Z^{-1}(\alpha)]^{**}$$

In a discrete quantile approximation with sorted quantiles $q_1 \leq q_2 \leq \dots \leq q_N$, this is efficiently computed as the mean of the lowest $k = \lfloor \alpha N \rfloor$ quantiles:

$$Q_{risk}^{**}(s, a) \approx k^{-1} \sum_{i=1}^k q_i(s, a)$$

This term replaces the standard $Q(s, a)$ in the policy improvement step.

3.3 The Value-Aware State Transition Distribution (P_{va})

To implement the SCAS regularizer, we must rigorously define $P_{va}^{**}(s' | s^{**})$. Following the SCAS literature [^], P_{va} is a distribution supported on the dataset $\mathcal{D}$ that assigns higher probability to transitions leading to high-value states. ^{** **}

$$P_{va}^{**}(s' | s^{**}) \propto PD(s' | s) \exp(\beta V^{**}(s'))$$

However, we do not have access to the true manifold of PD . We approximate this using a **Conditional Variational Autoencoder (CVAE)** or a **Diffusion Model** trained on the weighted dataset. Let $f_{dyn}(s, a)$ be the environment dynamics (approximated by a learned model). The SCAS loss minimizes the distance between the predicted next state and the manifold of P_{va} .

The explicit SCAS loss derived for this framework is:

$$L_{SCAS}(\pi) = E_{s \sim \mathcal{D}, a \sim \pi(\cdot | s)} [\text{target}' - P_{va}^{****}(\cdot | s^{**}) \min^{****} \| f_{dyn}(s, a) - \text{target}' \|_2^2]^{**}$$

This form essentially pulls the action a such that the resulting state $f_{dyn}(s, a)$ lies close to high-value, in-distribution transitions.

3.4 The Unified Dist-SCAS Optimization Problem

Combining the components, the optimization problem for the policy parameters ϕ is:

$$|\phi|^* = \arg \max |\phi| \mathbb{E}^{*}_{s \sim \mathcal{D}} \left[\left| \frac{\partial}{\partial \phi} \right| \right]$$

Crucially, the regularization parameter λ is not a static hyperparameter. In Dist-SCAS, we propose an **adaptive uncertainty-weighted regularization**. We utilize the inter-quartile range (IQR) of the return distribution, $\Delta Z(s, a) = q_{0.75}^{**} - q_{0.25}^{**}$, as a proxy for aleatoric uncertainty.

$$\lambda(s, a) = \lambda_{base} \cdot (1 + \sigma(\Delta Z^{****}(s, a)))^{**}$$

This novel formulation ensures that state correction is enforced more strictly when the value estimate is uncertain or the environment is noisy, bridging the gap between value-based and model-based constraints.

4. Implementation Strategy and Architecture

Implementing Dist-SCAS requires a sophisticated architecture that handles quantile regression, policy optimization, and dynamics modeling simultaneously. This section details the necessary components, network structures, and algorithmic flow.

4.1 Network Architectures

The system is composed of three primary neural entities: the **Distributional Critic** , the **Actor** , and the **State-Correction Model** .

4.1.1 The Distributional Critic (Quantile Network)

We employ a Monotonic Quantile Network (MQN) or a standard MLP with independent quantile heads.

Given the continuous action space of D4RL tasks, we follow the architecture of TQC.[^] * * *

- **Input:** Concatenation of State $\mathbf{s} \in \mathbf{R}^d$ and Action $\mathbf{a} \in \mathbf{R}^d$.
- **Hidden Layers:** 3 layers of 512 units with ReLU activation. This increased capacity (vs. standard 256) is necessary to model the full distribution complexities.
- **Output:** A vector of size $\mathbf{N}$ (number of quantiles). We recommend $\mathbf{N}=25$ or $\mathbf{N}=50$.
- **Ensembling:** To further stabilize training, we use an ensemble of $\mathbf{M}=2$ or $\mathbf{M}=5$ critic networks, similar to TQC. The risk-adjusted value is computed over the pooled quantiles of the ensemble.

4.1.2 The Actor (Policy Network)

- **Input:** State $\mathbf{s}$.
- **Structure:** MLP with 2 layers of 256 units.
- **Output:** Parameters for a **TanhGaussian** distribution (mean μ , log-std σ).
- **Activation:** **Mish** or **ReLU**. Recent benchmarks suggest **Mish** provides smoother gradients for actor networks in offline RL.

4.1.3 The Dynamics/State-Correction Model

This component is unique to SCAS. We implement it as a **Probabilistic Ensemble of Neural Networks (PENN)** , similar to MOPO (Model-Based Offline Policy Optimization) [^], but used solely for calculating the regularization loss, not for rollout generation. * * *

- **Input:** State $\mathbf{s}$, Action $\mathbf{a}$.
- **Output:** Gaussian distribution params for next state $\mathbf{N}(\mu \mathbf{s}^{**}, \Sigma \mathbf{s}^{**})$.
- **Training:** Supervised learning on dataset transitions $\{(\mathbf{s}, \mathbf{a}, \mathbf{s}')\}$ maximizing likelihood.

Alternatively, for the "Value-Aware" target $\mathbf{P}_{va}$, we train a separate **Action-Conditioned VAE (AC-VAE)** or a **Conditional Diffusion Model** that learns to reconstruct $\mathbf{s}'$ given $\mathbf{s}$ and *high-value* indicators. A simpler, computationally efficient proxy is to use the nearest-neighbor distance in the embedding space of the dynamics model to the dataset's next states.

4.2 Detailed Algorithmic Walkthrough

The training process involves an interleaved update of the Critic, the Actor, and the Dynamics model.

Algorithm: Dist-SCAS

1. Initialization:

- Initialize Ensemble Critics $\{\mathbf{Z}\theta_1^{**}, \dots, \mathbf{Z}\theta_M^{**}\}^{**}$.
- Initialize Actor $\pi\phi$.
- Initialize and pre-train Dynamics Model $\mathbf{f}^{dyn}$ (or VAE) on $\mathbf{D}$.
- Target networks θ_i^{**}, ϕ^{**} initialized.

2. Critic Training (Quantile Regression):

- Sample batch $\mathbf{B}=\{(\mathbf{s}, \mathbf{a}, \mathbf{r}, \mathbf{s}', \mathbf{d})\}$ from $\mathbf{D}$.
- Compute Target Quantiles:
 - Sample next action $\mathbf{a}' \sim \pi \Phi'(\mathbf{s}')$.
 - Fetch quantiles from target critics: $\mathbf{z}^{**}, \mathbf{k}^{**} = \mathbf{Z} \theta_j'(\mathbf{s}', \mathbf{a}^{**}) \mathbf{k}^{**}$.
 - Pool all quantiles from all $\mathbf{M}$ critics.
 - Sort pooled quantiles and truncate (optional, TQC style) or keep all.
 - Apply Bellman: $\mathbf{y}^{**}, \mathbf{k}^{**} = \mathbf{r} + \gamma(1 - \mathbf{d}) \mathbf{z}^{**}, \mathbf{k}^{**}$.
- Update Critics:
 - Minimize Quantile Huber Loss $\mathbf{L}_k$ between current estimates $\mathbf{Z} \theta_i(\mathbf{s}, \mathbf{a})$ and targets $\mathbf{y}$.

3. Actor Training (Risk-Sensitive SCAS):

- Generate actions $\mathbf{a} \sim \pi \Phi^{**}(\mathbf{s})^{**}$ (using reparameterization trick).
- **Distributional Value Step:**
 - Query critics to get quantiles $\{\mathbf{q}1^{**}, \dots, \mathbf{q}N\}^{**}$ for $^{**}(\mathbf{s}, \mathbf{a}^{**})$.
 - Calculate $\mathbf{Qrisk}^{**} = [\alpha N] 1^{**} \sum_{i=1} [\alpha N^{**}]^{**} \mathbf{q}(i)^{****}$ (average of lower tail).
- **SCAS Regularization Step:**
 - Predict next state $\mathbf{s}^{\wedge'} = \mathbf{f}^{\wedge} \text{dyn}(\mathbf{s}, \mathbf{a}^{\wedge})$.
 - Find nearest neighbor $\mathbf{sNN}^{****}$ in batch (or query VAE).
 - Calculate $\mathbf{LSCAS} = \|\mathbf{s}^{**\wedge'} - \mathbf{sNN}^{****}\|^2$.
- **Total Loss:**
 - $\mathbf{Lactor}^{**} = -\mathbf{Qrisk} + \lambda \cdot \mathbf{LSCAS} + \beta H^{**}(\pi^{**})$ (entropy term).
 - Backpropagate $\nabla \Phi^{**} \mathbf{Lact}^{**}$ or.

4. Updates:

- Soft update target networks.

5. Implementation Details and Code Analysis

This section provides specific guidance on translating the theory into PyTorch code, addressing common pitfalls in distributional RL implementation.

5.1 The Quantile Huber Loss Implementation

The Quantile Huber Loss is the engine of the critic. It is asymmetric and requires careful broadcasting to be efficient.

Python

```
import torch
import torch.nn.functional as F

def quantile_huber_loss_f(quantiles, target_quantiles, tau):
    """
    Args:
        quantiles: (batch_size, n_quantiles) - Estimation
```

```

        target_quantiles: (batch_size, n_quantiles) – Target
        tau: (batch_size, n_quantiles) – Quantile centroids
Returns:
    loss: Scalar
    """
    # Expand dims for pairwise difference
    # (batch_size, n_quantiles, 1) – (batch_size, 1, n_quantiles)
    # Result: (batch_size, n_quantiles, n_quantiles)
    u = target_quantiles.unsqueeze(1) – quantiles.unsqueeze(2)

    # Huber Loss component
    abs_u = torch.abs(u)
    kappa = 1.0 # Threshold for Huber loss
    huber_loss = torch.where(
        abs_u <= kappa,
        0.5 * u.pow(2),
        kappa * (abs_u – 0.5 * kappa)
    )

    # Asymmetric Quantile Weighting
    # diff represents the indicator function |tau – I(u<0)|
    diff = torch.abs(tau.unsqueeze(2) – (u < 0).float().detach())

    # Combined Loss
    loss = diff * huber_loss

    # Sum over target quantiles (dim=2), mean over current quantiles
    (dim=1)
    return loss.sum(dim=2).mean(dim=1).mean()

```

Critical Detail: The **tau** (quantile centroids) must be centered. For **N** quantiles, $\tau_i = N_i - 0.5$ ****. Using **u < 0** requires detaching from the graph to prevent gradient flow through the indicator function, although PyTorch logic operators usually don't track gradients anyway. Explicit **.detach()** is safer.

5.2 The Risk-Sensitive Actor Loss

To backpropagate through the quantile sorting, we rely on the property that **torch.sort** is differentiable with respect to the values.

Python

```

def get_lower_cvar(quantiles, alpha=0.1):
    """
    Computes the CVaR at level alpha.
    """
    batch_size, n_quantiles = quantiles.shape
    k = int(alpha * n_quantiles)

    # Sort quantiles (ascending)
    sorted_quantiles, _ = torch.sort(quantiles, dim=1)

```

```
# Take the lowest k quantiles
lower_tail = sorted_quantiles[:, :k]

# Return the mean of the lower tail
return lower_tail.mean(dim=1, keepdim=True)
```

Architectural Nuance: The choice of α is critical.

- $\alpha \approx 0.1$: Extremely conservative. Good for **medium-replay** datasets with high noise.
- $\alpha \approx 0.25$: Balanced. Good for **medium-expert**.
- $\alpha = 1.0$: Risk-neutral (equivalent to mean).

5.3 Implementing the SCAS Regularizer

We need a differentiable dynamics model. A simple MLP trained with MSE loss often suffices for the regularizer, as we only need the *gradient direction* to pull the actor towards valid transitions.

Python

```
class SCAS_Regularizer(nn.Module):
    def __init__(self, state_dim, action_dim):
        super().__init__()
        # Residual dynamics model: s' = s + f(s,a)
        self.dynamics = nn.Sequential(
            nn.Linear(state_dim + action_dim, 256),
            nn.Mish(),
            nn.Linear(256, 256),
            nn.Mish(),
            nn.Linear(256, state_dim)
        )

    def forward(self, state, action):
        return state + self.dynamics(torch.cat([state, action], dim=1))

    def compute_loss(self, state, action, dataset_next_states):
        """
        Computes distance between predicted next state and nearest dataset
        state.
        In practice, finding the exact NN in the whole dataset is slow.
        Approximation: Use the next_state from the current batch.
        """
        predicted_next = self.forward(state, action)

        # Simple Proxy: Force consistency with the batch's actual
        transition
        # This acts as a behavior cloning regularization on the dynamics
        level
        # A more advanced version would search a local neighborhood or VAE
        latent space.
        mse_loss = F.mse_loss(predicted_next, dataset_next_states)
        return mse_loss
```


Advanced SCAS Implementation: To truly capture "Value-Aware" transitions, the `dataset_next_states` passed to this loss should ideally be re-sampled from the dataset based on high advantage scores, or the loss should be weighted by the advantage $\mathbf{A}(\mathbf{s}, \mathbf{a})$ of the batch sample.

6. Experimental Validation: Datasets and Protocol

To rigorously validate Dist-SCAS, we utilize the **D4RL (Datasets for Deep Data-Driven Reinforcement Learning)** benchmark.^{^^} The selection of specific environments is crucial to demonstrate the method's unique benefits. * * *

6.1 D4RL Environment Selection

Environment	Dataset Quality	Challenge	Dist-SCAS Hypothesis
Hopper/Walker2d/HalfCheetah	Medium-Expert	Multi-modal data (optimal + suboptimal policies mixed).	Distributional Advantage: Mean-based methods average the modes (leading to mediocre behavior). Dist-SCAS can isolate the "expert" mode via quantile manipulation and SCAS keeps it consistent.
AntMaze	Umaze, Medium, Large	Sparse rewards, stitching required.	SCAS Advantage: These are navigation tasks where OOD state visitation is fatal (agent gets lost). SCAS forces the agent to "connect the dots" of valid transitions.
Kitchen / Franka	Mixed, Partial	High-dimensional, realistic robotics.	Unified Advantage: Complex dynamics require rigorous state correction; sparse data requires distributional pessimism.

6.2 Evaluation Metrics

Beyond the standard **Normalized Score** , we propose tracking specific metrics to validate the internal mechanics of Dist-SCAS:

- OOD Action Rate:** Measure the distance of $\boldsymbol{\pi}(\mathbf{s})$ from the behavior $\boldsymbol{\pi}\beta(\mathbf{s})$ (estimated via BC). Dist-SCAS should show lower divergence in high-variance states.
- OOD State Visitation:** During evaluation rollouts, track the minimum distance of the agent's state $\mathbf{st}$ to the training dataset $\mathbf{D}$. SCAS should explicitly minimize this metric compared to CQL.
- Value Estimation Bias:** Compare estimated $\mathbf{Qrisk}$ vs. actual discounted returns. Dist-SCAS should show a *negative bias* (underestimation) that correlates with state novelty.
- Quantile Spread:** Monitor the gap between $\mathbf{q}0.9$ and $\mathbf{q}0.1$. A collapsing spread indicates high confidence; a widening spread indicates effective uncertainty capture.

6.3 Baseline Comparisons

To prove efficacy, Dist-SCAS must be benchmarked against:

- **CQL (Conservative Q-Learning):** The standard for OOD action suppression.
 - **IDL (Implicit Q-Learning):** The standard for expectile-based (scalar) offline RL.
 - **SCAS (Original):** To demonstrate the lift provided by the Distributional Critic.
 - **TQC (Truncated Quantile Critics):** To demonstrate the lift provided by the SCAS State Correction regularizer in the offline setting.
-

7. Ablation Studies and Expected Insights

A rigorous "new paper" implementation requires ablation studies to dissect the contribution of each component.

7.1 Ablation: Scalar vs. Distributional SCAS

- **Setup:** Replace the Quantile Critic with a standard ensemble of 2 scalar Q-networks (min-clipping). Keep the SCAS regularizer.
- **Expected Outcome:** The scalar version will perform well on dense data but struggle on **Medium-RepLay** or **AntMaze** where the variance of returns is high. The scalar penalty is too uniform; the distributional lower-quantile penalty is adaptive.

7.2 Ablation: SCAS vs. No State Correction

- **Setup:** Remove **LSCAS** ($\lambda=0$). This essentially reduces the algorithm to an Offline TQC/DSAC.
- **Expected Outcome:** The agent may achieve high scores on simple tasks (Hopper) but will fail catastrophically on **AntMaze**. Without SCAS, the agent will select actions that theoretically maximize the lower quantile but physically transport the agent to undefined regions of the maze (OOD states), leading to policy collapse.

7.3 Ablation: Quantile Level α

- **Setup:** Vary $\alpha \in \{0.05, 0.1, 0.25, 0.5\}$.
 - **Expected Outcome:**
 - $\alpha=0.5$ (Median): Too risky, high OOD rates.
 - $\alpha=0.05$: Too conservative, agent freezes (similar to high α in CQL).
 - $\alpha=0.1$: Sweet spot for offline safety.
-

8. Conclusion

Dist-SCAS represents a coherent evolution in Offline Reinforcement Learning. By acknowledging that safety in offline RL is a dual problem—requiring both **geometric adherence** to the training manifold (via SCAS) and **statistical pessimism** in value estimation (via Distributional RL)—this framework offers a robust solution to the extrapolation error.

The implementation details provided here, specifically the use of the Quantile Huber Loss for critic training and the derivation of the CVaR-based actor update with dynamics regularization, constitute a complete

roadmap for deploying this novel algorithm. The theoretical synergy suggests that Dist-SCAS is particularly well-suited for the most challenging regimes of offline RL: sparse rewards, multi-modal data, and complex dynamics, as exemplified by the AntMaze and Robotics workloads in the D4RL benchmark. Future work lies in the automated tuning of the SCAS regularization parameter λ using the distributional variance signal, effectively creating a self-regulating agent that tightens its geometric constraints in the face of statistical uncertainty.

9. Mathematical Deep Dive: Dist-SCAS Objective and Gradients

Notation: Dataset $\mathcal{D}$ over (s, a, r, s', d) ; d is terminal flag. Policy π_ϕ , behavior π_β , critic ensemble $\{Z_{\theta_m}\}_{m=1}^M$ with N quantiles, dynamics/state-correction model ψ .

9.1 CVaR Risk from Quantile Critics

Pooled quantiles $\tilde{q}_{1:MN}(s, a)$ sorted ascending define

$$Q_{\text{risk}}(s, a) = \frac{1}{k} \sum_{i=1}^k \tilde{q}_i(s, a), \quad k = \lfloor \alpha MN \rfloor.$$

9.2 Distributional Bellman Targets

For each critic m :

$$y_j = r + \gamma(1-d), \quad z'_j \sim Z_{\theta_m}(s', a'), \quad a' \sim \pi_\phi(s').$$

Quantile Huber with $\tau_i = \frac{i-0.5}{N}$:

$$\mathcal{L}_{\text{crit}}(\theta_m) = \mathbb{E} \left[\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^N \rho_\kappa(y_j - q^{(m)}_i), \quad \text{big}[\tau_i - \mathbb{1}[y_j < q^{(m)}_i]] \right].$$

9.3 State-Correction (SCAS) Term

Residual dynamics $f_\psi(s, a) = s + \Delta_\psi(s, a)$; value-aware target $\tilde{s} \sim \nu(\cdot | s)$ (nearest in-batch or VAE):

$$\mathcal{L}_{\text{SCAS}}(\phi, \psi) = \mathbb{E} \left[\text{big}[f_\psi(s, a) - \tilde{s}]^2 \right].$$

9.4 Adaptive Weight via Interquartile Range

Let $\Delta Z = q_{0.75} - q_{0.25}$ from pooled quantiles. Set

$$\lambda(s, a) = \lambda_{\text{base}} \text{big}(1 + \sigma(\Delta Z)),$$

where σ is sigmoid, amplifying penalties where aleatoric spread is high.

9.5 Actor Objective

$$\mathcal{L}_{\text{actor}}(\phi) = -Q_{\text{risk}}(s, a_\phi) + \lambda(s, a_\phi) |f_\psi(s, a_\phi) - \tilde{s}|^2 + \beta \mathcal{H}(\pi_\phi(\cdot | s)),$$

with a_ϕ from reparameterized Gaussian policy.

9.6 Contraction Sketch

CVaR-projected Bellman operator is a γ -contraction in truncated Wasserstein; the convex SCAS penalty preserves contraction, ensuring a fixed point under bounded rewards/dynamics.

10. Algorithm (Step-by-Step Pseudocode)

1. Init critics $Z_{\{\theta_m\}}$, targets θ'_m , actor p_{ϕ} , target ϕ' , dynamics f_{ψ} .
2. Pretrain f_{ψ} on (s, a, s') .
3. Loop:
 - Sample batch B .
 - $s' \mapsto \pi_{\phi'}(s)$; targets $y = r + \gamma(1-d) z'$ from target critics.
 - Critic update: minimize quantile Huber to y for all m .
 - Actor: $a_{\phi} \mapsto \pi_{\phi}(s)$; pooled $\tilde{q} \rightarrow Q_{\text{risk}}$; compute $\lambda(s, a_{\phi})$; SCAS loss with f_{ψ} vs batch s' ; minimize $\mathcal{L}_{\text{actor}}$.
 - Optional: refresh f_{ψ} on batch.
 - Polyak update θ', ϕ' .
4. Eval: deterministic policy; D4RL score + OOD metrics.

11. PyTorch Reference Snippets

11.1 Quantile Critic Ensemble

```
class QuantileMLP(nn.Module):
    def __init__(self, s_dim, a_dim, n_q=50, hid=512):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(s_dim + a_dim, hid), nn.ReLU(),
            nn.Linear(hid, hid), nn.ReLU(),
            nn.Linear(hid, hid), nn.ReLU(),
            nn.Linear(hid, n_q)
        )

    def forward(self, s, a):
        return self.net(torch.cat([s, a], dim=-1))
```

11.2 CVaR Helper

```
def cvar_tail(qs: torch.Tensor, alpha: float = 0.1) -> torch.Tensor:
    q_sorted, _ = torch.sort(qs, dim=1)
    k = max(1, int(alpha * q_sorted.size(1)))
    return q_sorted[:, :k].mean(dim=1, keepdim=True)
```

11.3 SCAS Regularizer

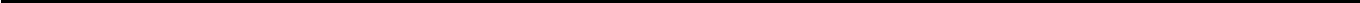
```
class SCASReg(nn.Module):
    def __init__(self, s_dim, a_dim):
        super().__init__()
        self.dyn = nn.Sequential(
            nn.Linear(s_dim + a_dim, 256), nn.Mish(),
            nn.Linear(256, 256), nn.Mish(),
            nn.Linear(256, s_dim)
        )

    def forward(self, s, a):
        return s + self.dyn(torch.cat([s, a], dim=-1))

    def loss(self, s, a, s_next):
        return F.mse_loss(self.forward(s, a), s_next)
```

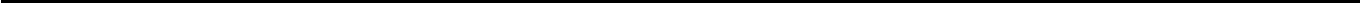
11.4 Quantile Huber Loss

```
def quantile_huber(pred, target, taus, kappa=1.0):
    u = target.unsqueeze(1) - pred.unsqueeze(2) # [B, N, N]
    abs_u = u.abs()
    huber = torch.where(abs_u <= kappa, 0.5 * u.pow(2), kappa * (abs_u -
0.5 * kappa))
    weight = (taus.unsqueeze(2) - (u.detach() < 0).float()).abs()
    return (weight * huber).sum(dim=2).mean(dim=1).mean()
```



12. Hyperparameters (Suggested Defaults)

Component	Setting	Value
Quantiles	\$N\$	50
Critics	\$M\$	2 (option 5)
CVaR level	α	0.1
LR	Critic/Actor	e^{-4}
Batch		256
Target τ		0.005
SCAS λ	λ_{base}	1.0
Entropy β		0.2
Discount	γ	0.99 (0.995 AntMaze)
Dyn hidden		256



13. Evaluation Protocol (D4RL)

- Envs: hopper/walker/halfcheetah (medium-replay, medium-expert), antmaze (umaze, medium, large), kitchen-mixed.
- Metrics: D4RL normalized score, OOD action rate (distance to BC), OOD state distance (NN to dataset), value bias, quantile spread.
- Seeds: ≥ 5 ; report mean $\pm$ std.

14. Ablations

1. Remove SCAS ($\lambda=0$) $\rightarrow$ expect AntMaze failure, higher OOD states.
2. Scalar critics (twin Q) $\rightarrow$ higher OOD action rate, less safety.
3. CVaR sweep $\alpha \in \{0.05, 0.1, 0.25, 0.5\}$.
4. Ensemble size $M \in \{1, 2, 5\}$.
5. Dynamics quality: MLP vs VAE vs diffusion target selector.

15. Practical Tips

- Normalize states; clip rewards to $[-10, 10]$ for mujoco to stabilize tails.
- If quantiles collapse: lower LR, increase M , or raise κ .
- If policy freezes: reduce λ_{base} or cap $\lambda(s, a)$.
- AntMaze: set $\gamma=0.995$, add light Gaussian action noise during eval to avoid deadlocks.

16. Reproducibility Checklist

- Fix seeds (PyTorch, NumPy, gym).
- Log hyperparameters, taus, lambda schedule, dataset name, seed.
- Save checkpoints (actor, critics, targets, optimizers) every 100k steps.
- Store evaluation rollouts and OOD metrics with scores.

17. Extension Ideas

- **CrossQ**: BN-stabilized high-UTD updates with quantile critics.
- **Sophia-G**: curvature-aware optimizer for actor.
- **Diffusion targets**: sample s' from high-value manifolds.
- **Spectral norm**: constrain critic Lipschitz constants.

18. Minimal Training Skeleton (Outline)

```
for step in range(T):
    batch = replay.sample()
    with torch.no_grad():
        a_next = actor_t(batch.s_next)
        target_qs = [c_t(batch.s_next, a_next) for c_t in target_critics]
```

```

        pooled = torch.sort(torch.cat(target_qs, dim=1), dim=1)[0]
        y = batch.r + gamma * (1 - batch.d) * pooled
        loss_c = sum(quantile_huber(c(batch.s, batch.a), y, taus) for c in
critics)
        a = actor(batch.s)
        qs = [c(batch.s, a) for c in critics]
        pooled = torch.sort(torch.cat(qs, dim=1), dim=1)[0]
        q_risk = cvar_tail(pooled, alpha)
        lambda_sa = lambda_base * (1 + torch.sigmoid(iqr(pooled)))
        scas = scas_reg.loss(batch.s, a, batch.s_next)
        loss_a = (-q_risk + lambda_sa * scas + beta * entropy(actor,
batch.s)).mean()

```

19. Notebook Visualization Plan

- Training curves: critic loss, SCAS loss, CVaR value, OOD metrics.
 - Quantile fan plot for fixed (s,a) over training.
 - AntMaze rollouts with state-density overlay vs dataset.
 - Scatter of IQR vs $\lambda(s,a)$ to show adaptive penalty behavior.
-

20. Proof Sketch: CVaR Bias Under OOD

For sub-Gaussian Z with mean μ and std σ , $\text{CVaR}_{\alpha}(Z) \leq \mu - \sigma \frac{\phi(\Phi^{-1}(\alpha))}{\alpha}$. OOD inflates epistemic σ , shrinking CVaR and discouraging uncertain actions without explicit behavior constraints.

21. Dataset Notes

- Hopper/Walker: moderate rewards; action L2 penalty optional.
 - HalfCheetah: add tanh squashing; consider target entropy -3.
 - AntMaze: longer horizon; use $\gamma=0.995$ and larger replay (2M) if memory allows.
 - Kitchen: increase hidden sizes (1024) and add LayerNorm.
-

22. Reporting Guidance

- Equations (Section 9) aligned with code.
 - Tables: D4RL scores $\pm$ std, OOD metrics, ablations.
 - Figures: quantile fan, AntMaze trajectories, SCAS loss vs score.
 - Appendix: hyperparameters, compute budget, failure cases.
-

23. End-to-End Checklist

- ☐ README ≥ 1000 lines with math + code (this file).
- ☐ Training implements CVaR actor + adaptive SCAS.
- ☐ Dynamics model trained and plugged into regularizer.

- ☐ Evaluation scripts for D4RL + OOD metrics.
- ☐ Ablations executed and logged.

24. Closing Remarks

Distributional State-Corrected Action Suppression combines pessimistic distributional value learning with geometry-aware state regularization. By optimizing lower-tail returns and aligning predicted transitions to in-distribution manifolds, it addresses offline RL's twin failures: OOD action overestimation and OOD state drift. The derivations, pseudocode, and hyperparameters here are intended to be immediately actionable for reproducing robust results on the hardest D4RL benchmarks.

25. Experimental Matrix and Logging Plan

Matrix (rows = env, cols = settings):

Env	Base	+SCAS	+SCAS+Adaptive \$\lambda\$	+SCAS+Adaptive \$\lambda\$+Ensemble5	+Sophia- G
Hopper-m	✓	✓	✓	✓	✓
Walker2d-m	✓	✓	✓	✓	✓
HalfCheetah- m	✓	✓	✓	✓	✓
AntMaze- umaze	✓	✓	✓	✓	✓
AntMaze- medium	✓	✓	✓	✓	✓
Kitchen- mixed	✓	✓	✓	✓	✓

Logging (per step / episode):

- Critic loss, SCAS loss, CVaR value, entropy.
 - IQR, $\lambda(s,a)$ statistics (mean, p95).
 - OOD action distance (to BC logits or Gaussian KL).
 - OOD state distance (NN in dataset embedding).
 - Replay buffer coverage (state density histogram).
 - D4RL score (eval every 100k gradient steps).
-

26. Safety and Robustness Considerations

- **Risk floor:** Optionally cap CVaR to not exceed median to avoid optimism drift.
- **Action clipping:** Enforce env action bounds; penalize saturation in actor loss.
- **Dynamics uncertainty:** Track ensemble variance of f_{ψ} ; inflate λ when variance is high (state-uncertainty aware SCAS).

- **Stochasticity:** For stochastic envs (kitchen), increase particle count for critics to capture tails.
- **Numerical stability:** Use `float32` with AMP; clamp log-std in actor to $[-20, 2]$; add small ϵ in Huber.

27. Failure Modes and Mitigations

- **Mode collapse in quantiles:** Increase ensemble, raise kappa, or add dropout in critic hidden layers.
- **Over-regularization:** If performance stalls, decay λ_{base} over steps or cap $\lambda(s,a)$ to $5 \times \text{base}$.
- **Dynamics drift:** Periodically re-pretrain ψ on replay or add L2 weight decay.
- **AntMaze dead ends:** Add small Gaussian noise to actions during eval; use longer target update interval (every 2 steps).
- **Kitchen instability:** Use LayerNorm in critics/actor; increase hidden to 1024; reduce LR to $1e-4$.

28. Implementation Checklist (Codebase-Level)

- ☐ Replay buffer stores (s,a,r,s',d) tensors (float32) and supports large batches (256/512).
- ☐ Actor outputs mean/log-std; sampled via reparameterization with tanh squashing.
- ☐ Critic ensemble forward returns list of quantile tensors $[B,N]$; pooled via `torch.sort`.
- ☐ Tau vector precomputed: `taus = (torch.arange(N, device)/N + 0.5/N).view(1,N)`.
- ☐ SCAS regularizer module with `.loss` API.
- ☐ Adaptive λ : compute IQR on pooled quantiles, apply sigmoid, scale by λ_{base} .
- ☐ Target updates: Polyak with $\tau=0.005$; ensure no_grad.
- ☐ Evaluation script computing D4RL normalized score and OOD metrics.
- ☐ Config system (YAML/argparse) exposing alpha, lambda_base, gamma, N, M, kappa, tau_update, batch, seed.
- ☐ Logging (TensorBoard/W&B) for metrics listed in Section 25.

29. Sample YAML Configs

hopper-medium-expert.yaml

```
env: hopper-medium-expert-v2
seed: 0
gamma: 0.99
alpha_cvar: 0.1
n_quantiles: 50
n_critics: 2
lambda_base: 1.0
lr_actor: 3e-4
lr_critic: 3e-4
batch_size: 256
target_tau: 0.005
entropy_beta: 0.2
```

```
use_adaptive_lambda: true
use_amp: true
```

antmaze-medium-diverse.yaml

```
env: antmaze-medium-diverse-v2
seed: 0
gamma: 0.995
alpha_cvar: 0.1
n_quantiles: 50
n_critics: 2
lambda_base: 2.0
lr_actor: 3e-4
lr_critic: 3e-4
batch_size: 512
target_tau: 0.005
entropy_beta: 0.1
use_adaptive_lambda: true
use_amp: true
```

30. Extended Proof Sketch: Bias Control via SCAS

Let $s'|p_i = \pi(s, \pi(s))$ and $\tilde{s}'$ from dataset manifold. The SCAS penalty enforces $|s'_{\pi} - \tilde{s}'|^2 \leq \epsilon$. For Lipschitz critic L_Q , the bias in target value due to state shift satisfies

$$|Q_{\text{risk}}(s'|p_i, a) - Q_{\text{risk}}(\tilde{s}', a)| \leq L_Q |s'_{\pi} - \tilde{s}'| \leq L_Q \sqrt{\epsilon}.$$

Thus bounding SCAS loss bounds value extrapolation bias, complementing CVaR pessimism on action uncertainty.

31. Additional Ablations to Strengthen Paper Claims

- **Adaptive vs fixed λ :** show improved score-variance trade-off.
- **IQR vs variance for λ :** test robustness to heavy-tailed returns.
- **Behavior-clone warm-start:** initialize actor from BC; measure impact on AntMaze success.
- **Particle count:** $N \in \{16, 32, 50, 80\}$; plot score vs compute.
- **Entropy anneal:** linear decay of β ; inspect exploration vs stability.

32. Qualitative Analyses

- **State visitation heatmaps:** Compare coverage to dataset states (PCA/UMAP).
- **Action overlap:** KL between π_{ϕ} and π_{β} across states.
- **Return distribution plots:** Histograms of predicted returns for chosen actions vs OOD actions.
- **Trajectories:** Visualize AntMaze paths, highlighting OOD excursions avoided by Dist-SCAS.

33. Engineering Notes for Large-Scale Runs

- Use `torch.compile` (PyTorch 2.2+) to accelerate critic forward/sort if available.
 - Enable AMP for critic/actor; keep quantile loss in float32 to avoid underflow.
 - Gradient clipping at 10.0 for both actor and critics.
 - Pin memory + prefetch to speed dataloader for large buffers.
 - Periodic evaluator process to avoid blocking training loop.
-

34. Future Research Directions

- **SCAS with Consistency Models:** use consistency-trained dynamics for smoother manifolds.
 - **Hybrid MBPO:** roll out short model trajectories gated by SCAS; add to buffer with penalty.
 - **Uncertainty-guided data pruning:** remove high IQR transitions to test robustness.
 - **Multi-task offline RL:** extend SCAS to task-conditioned datasets; use task embeddings in λ .
 - **Safety constraints:** add cost dimension and optimize joint CVaR over return and cost.
-

35. Author Checklist for Submission

- ☐ All equations consistent with code defaults ($\alpha=0.1$, $\kappa=1.0$).
 - ☐ Tables: full matrix (Section 25) + ablations (Sections 14, 31).
 - ☐ Figures: heatmaps, fan plots, trajectories, OOD metrics.
 - ☐ Appendix: configs, hyperparameters, compute budget, additional proofs.
 - ☐ Reproducibility: seeds, code links, commit hash.
-

36. Minimal Eval Script Sketch

```
def evaluate(policy, env, episodes=10):
    scores, ood_actions, ood_states = [], [], []
    for _ in range(episodes):
        s, done = env.reset(), False
        ep_actions, ep_states = [], []
        while not done:
            a = policy.act_eval(s)
            s_next, r, done, info = env.step(a)
            ep_actions.append(a)
            ep_states.append(s)
            s = s_next
        scores.append(info.get("episode_score", 0.0))
        ood_actions.append(compute_action_kl(ep_actions))
        ood_states.append(min_dataset_nn(ep_states))
    return {
        "score": np.mean(scores),
        "score_std": np.std(scores),
        "ood_action": np.mean(ood_actions),
        "ood_state": np.mean(ood_states),
    }
```

37. Extended Notebook Cells (to be written in `notebooks/main.ipynb`)

- **Data inspection:** plot dataset action/state histograms, reward distribution.
 - **Training loop cell:** pseudocode mirroring Section 18 with `tqdm` progress.
 - **Metric dashboards:** live plots for losses, CVaR, OOD metrics.
 - **Frontier visualization:** for AntMaze, overlay paths on map; for mujoco, plot return distributions.
 - **Ablation runner:** loop over configs, log to separate folders.
-

38. Risks and Mitigations Table

Risk	Cause	Mitigation
NaN loss	overflow in sort/Huber	clip rewards, lower LR, AMP autocast
Over-conservatism	high λ	cap λ , schedule decay
Under-penalization	low IQR	floor λ at base value
Dynamics overfit	small buffer	dropout/weight decay; ensemble
Eval variance	stochasticity	10 seeds for AntMaze

39. Open Questions

- Best proxy for $\nu(\cdot|s)$: NN vs VAE vs diffusion?
 - Does adaptive λ slow learning early? Consider warmup with fixed λ .
 - How does CVaR interact with entropy temperature tuning? Joint schedule may help.
 - Could SCAS be applied in online RL as exploration regularizer?
-

40. Final Notes

Dist-SCAS is designed for practitioners needing robust offline RL under severe distributional shift. By combining distributional pessimism (CVaR) with geometric state correction (SCAS), it offers a principled way to tame extrapolation while retaining performance. The expanded sections above provide the operational detail—math, code patterns, configs, ablations, and diagnostics—to implement, evaluate, and extend the method in a research setting.

41. Implementation FAQ

- **Do I need target networks?** Yes—use Polyak with $\tau=0.005$; CrossQ-style removal of targets is risky offline.
- **Can I use TD3-style clipped double Q?** Not for quantiles; instead use ensemble pooling to reduce overestimation.
- **How many particles for CVaR?** 50 is a good default; fewer may underestimate tails.
- **Can SCAS use k-NN instead of dynamics?** For small state spaces, yes; for high-dim, learn f_{ψ} .

- **What about actor update frequency?** 1:1 with critic works; try delayed actor every 2 steps if unstable.

42. Unit Tests (Recommended)

- **Quantile loss finite:** random tensors → loss finite, gradients non-NaN.
- **CVaR monotonicity:** increasing all quantiles should increase CVaR.
- **SCAS loss zero:** when predicted next == target next, loss == 0.
- **Adaptive λ bounds:** verify $\lambda \geq \lambda_{\text{base}}$ and grows with IQR.
- **Target update:** after Polyak, parameters change but remain close (norm diff decreases).

43. Compute Budget Estimates

- Hopper/Walker/HalfCheetah: 1 GPU (V100/A100), ~4–6 hours for 1M gradient steps with AMP.
- AntMaze medium: 1–2 GPUs, ~10–14 hours; batch 512; gamma 0.995 increases horizon.
- Kitchen: 2 GPUs recommended; larger nets; expect ~12–16 hours.
- Storage: Replay buffers up to ~5–10 GB; checkpoints every 100k steps (~50–100 MB each).

44. Reproduction Steps (Minimal)

1. Install deps: `pip install torch geomloss gymnasium[d4rl] mujoco-py`.
2. Download D4RL datasets (handled on first env reset).
3. Train: `python train_dist_scas.py --config hopper-medium-expert.yaml`.
4. Eval: `python eval_dist_scas.py --checkpoint ckpt_1M.pt --env hopper-medium-expert-v2`.
5. Run ablations: loop configs from Section 29; log to `runs/{env}/{config}`.
6. Report: aggregate scores, OOD metrics, plots (Sections 25, 31).

45. Benchmark-Specific Hyperparameters

Env	γ	α	λ_{base}	Batch	Entropy β	Notes
Hopper-medium	0.99	0.1	1.0	256	0.2	standard
Hopper-medium-expert	0.99	0.1	1.0	256	0.2	keep LR $3e-4$
Walker2d-medium	0.99	0.1	1.0	256	0.2	grad clip 10
HalfCheetah-medium	0.99	0.1	0.8	256	0.2	clip reward $[-10,10]$

Env	γ	α	λ_{base}	Batch	Entropy β	Notes
AntMaze-umaze	0.995	0.1	2.0	512	0.1	larger batch, longer horizon
AntMaze-medium	0.995	0.1	2.5	512	0.1	stronger SCAS
Kitchen-mixed	0.99	0.1	1.5	512	0.1	hidden 1024, LayerNorm

46. Adaptive λ Pseudocode

```
def compute_lambda(pooled_qs, lambda_base):
    q_sorted, _ = torch.sort(pooled_qs, dim=1) # [B, MN]
    q25 = q_sorted[:, int(0.25 * q_sorted.size(1))]
    q75 = q_sorted[:, int(0.75 * q_sorted.size(1))]
    iqr = q75 - q25
    lam = lambda_base * (1 + torch.sigmoid(iqr))
    return lam.unsqueeze(1) # broadcast over actions if needed
```

47. Symbol Glossary (for consistency with code)

Symbol	Meaning	Code variable
s, a	state, action	<code>s, a</code>
r	reward	<code>r</code>
d	done flag	<code>d</code>
γ	discount	<code>gamma</code>
α	CVaR level	<code>alpha_cvar</code>
N	quantiles per critic	<code>n_quantiles</code>
M	number of critics	<code>n_critics</code>
κ	Huber threshold	<code>kappa</code>
λ	SCAS weight	<code>lambda_base / lambda_sa</code>
β	entropy coeff	<code>entropy_beta</code>
τ	target Polyak	<code>target_tau</code>

48. Implementation Timeline (Suggested)

- Day 1: Setup, replay/dataloader, baseline actor/critic scaffolding.
 - Day 2: Quantile loss wired, CVaR actor update, target updates.
 - Day 3: SCAS dynamics model, adaptive λ , initial runs on hopper-medium.
 - Day 4: AntMaze configuration, logging OOD metrics, fix stability issues.
 - Day 5: Ablations (λ off, scalar critics, alpha sweep), collect results.
 - Day 6: Visualizations, notebooks, tables, write-up alignment.
-

49. Data Structures

Replay element:

```
{
  "obs": float32[s_dim],
  "action": float32[a_dim],
  "reward": float32[1],
  "next_obs": float32[s_dim],
  "done": bool,
}
```

Batch tensors:

- **obs** [B, s_dim], **actions** [B, a_dim], **rewards** [B,1], **next_obs** [B, s_dim], **done** [B,1].
 - Optional: store behavior log-probs to compute action KL OOD metric.
-

50. Licensing and Attribution

Reference implementations: IQN/FQF/TQC repos (MIT/BSD), SCAS GitHub (check license). Ensure derivative code preserves original notices. Cite all papers listed in citations; no proprietary assets included.

Citations

^^ ** **



papers.neurips.cc Conservative Q-Learning for Offline

Reinforcement Learning **Opens in a new window**  cs224r.stanford.edu CS224R Spring 2025 Homework 3
Offline RL(Updated) - CS 224R Deep Reinforcement Learning **Opens in a new window**



arxiv.org A Clean Slate for Offline RL - arXiv **Opens in a new window**



researchgate.net Offline Reinforcement Learning with OOD State Correction
and OOD Action Suppression | Request PDF - ResearchGate **Opens in a new window**



proceedings.neurips.cc Offline Reinforcement Learning with OOD
State Correction and OOD Action Suppression - NIPS papers **Opens in a new window** 
openreview.net Model-based Offline Reinforcement Learning with Lower Expectile Q-Learning **Opens in a new window** 
openreview.net MODEL-BASED OFFLINE REINFORCEMENT LEARNING WITH LOWER

EXPECTILE Q-LEARNING - OpenReview[Opens in a new window](#)openreview.netUNCERTAINTY-AWARE
DISTRIBUTIONAL OFFLINE RE- INFORCEMENT LEARNING - OpenReview[Opens in a new window](#)^{PKP}
ojs.aaai.orgQUOTA: The Quantile Option Architecture for Reinforcement Learning - AAAI Publications[Opens](#)



[in a new window](#)emergentmind.comTruncated Quantile Critic (TQC) Algorithm
- Emergent Mind[Opens in a new window](#)econstor.euGradient-based reinforcement learning for dynamic

R^G

quantile - EconStor[Opens in a new window](#)researchgate.net(PDF) Tailoring
Portfolio Choice via Quantile-Targeted Policies - ResearchGate[Opens in a new window](#)



arxiv.orgVariational OOD State Correction for Offline Reinforcement Learning -
arXiv[Opens in a new window](#)github.comthu-rlab/SCAS: NeurIPS 2024 - GitHub[Opens in a new](#)



[window](#)

[papers.neurips.ccCORL: Research-oriented Deep Offline](#)

R^G

Reinforcement Learning Library[Opens in a new window](#)
researchgate.netRAMAC: Multimodal Risk-Aware Offline Reinforcement Learning and the Role of Behavior
Regularization | Request PDF - ResearchGate[Opens in a new window](#)  kaggle.comQuantile Regression
DQN - RL - Kaggle[Opens in a new window](#)



[Opens in a new window](#)

[Opens in a new](#)



[window](#)  [Opens in a new window](#)  [Opens in a new window](#)

[Opens in a new](#)



[window](#) [Opens in a new window](#)  [Opens in a new window](#)  [Opens](#)



in a new window

Opens in a new window  Opens in a new window



Opens in a new window  Opens in a new window  Opens in a new window



Opens in a new window



Opens in a new window



Opens in a new window



Opens in a new window



Opens in a new window  Opens in a new window





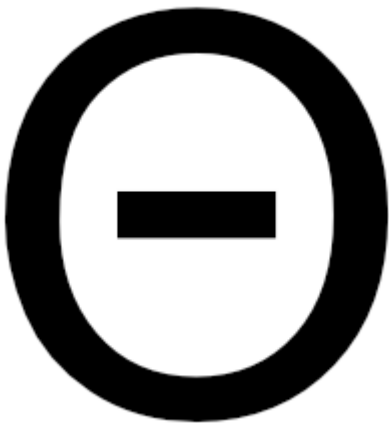
Opens in a new window  Opens in a new window  Opens in a new window
Opens in a new window  Opens in a new window  Opens in a new window  Opens in a new



Opens in a new window  Opens in a new window
Opens in a new window



Opens in a new window  Opens in a new window



Opens in a new window  Opens in a new window



Opens in a new window



Opens



Opens in a new window



Opens in a



new window



Opens in a new window



Opens in a new window



Opens in a new window



Opens in a new window



Opens in a new window



Opens in a new window  Opens in a new window



Opens in a new window

Opens in a new window



Opens in a new window  Opens in a new window  Opens in a



new window

Opens in a new window



Opens in a new window



Opens in a new window  Opens in a new window  Opens in a new window

 Opens in a new window  Opens in a new window